



- ▶ Alfabetos, cadenas y lenguajes
- ▶ Sistema binario
- ▶ Representación de números reales en la computadora

Bibliografía

- *Fundamentos de programación. sec. 1.3.3 y 1.61*, Luis Joyanes.
- *Introducción a la informática. sec. 2.1 y 2.2*, George Beekman.
- *Introducción a la programación con Python. sec 1.2*, Andrés Marzal, Isabel Gracia y Pedro García.
- *Computational Physics, problems solved with Python. sec 2.1 y 2.2*, Rubin H. Manuel J. Páez and Cristian C. Bordeianu.
- *Teoría de la Computación**, Rodrigo de Castro Korgi.

Alfabetos, cadenas y lenguajes

Desde nuestra infancia comenzamos a usar el lenguaje, mediante el uso de símbolos. En español (el idioma colonial en nuestra región), por ejemplo, el abecedario

$$\{a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z\}$$

forma un conjunto no vacío cuyos elementos (27 en total) se llaman símbolos. Se puede construir una teoría formal matemáticamente basada en estos conceptos y utilizando la teoría de conjuntos. Aquí solo mencionaremos un par de conceptos importantes para entender el sistema binario. La persona interesada en los detalles puede consultar el texto de Rodrigo de Castro Korgi.

Definición: Alfabeto

Un **alfabeto** es un conjunto finito no vacío cuyos elementos se llaman **símbolos**. De manera arbitraria, un alfabeto se denota por la letra griega Σ .

Ejemplo: Sea $\Sigma = \{a, b\}$ el alfabeto formado por dos símbolos, a y b . Podemos construir una secuencia finita sobre Σ de la siguiente manera: $aab, aba, bbabab$. A cada una de las secuencias finitas sobre Σ se le llama **cadenas** o **palabras** sobre Σ . La **longitud** de una cadena, u , es el número de símbolos que la componen y se denota como $|u|$. Por ejemplo, $|aba| = 3$. $|bbabab| = 6$. Note que los símbolos repetidos también se cuentan. Al conjunto de todas las cadenas sobre un alfabeto se denota como Σ^* . En nuestro ejemplo anterior, $\Sigma^* = \{\lambda, a, b, aab, aba, bbabab, \dots\}$, donde λ es la cadena vacía.

Ejemplo: $\Sigma = \{0, 1\}$ alfabeto binario. Cadenas: 0101, 1100, 00101.

$$\Sigma^* = \{\lambda, 0, 1, 0101, 1100, 00101, \dots\}$$

Ejemplo: $\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ alfabeto decimal. Cadenas: 10, 20, 100, 200, 220, ...

$$\Sigma^* = \{\lambda, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 20, 100, 200, 220, \dots\}$$

Ejemplo: $\Sigma = \{|\uparrow\rangle, |\downarrow\rangle\}$ alfabeto en computación cuántica. Cadenas: $|\uparrow\rangle, |\downarrow\rangle, |\uparrow\rangle|\downarrow\rangle, |\downarrow\rangle|\uparrow\rangle, \dots$
Un estado de qubit es una superposición de los estados base $|\uparrow\rangle$ y $|\downarrow\rangle$:

$$|\psi\rangle = \alpha|\uparrow\rangle + \beta|\downarrow\rangle$$

Finalmente, sobre el alfabeto podemos construir un **lenguaje** L como un subconjunto de Σ^* . Es decir, $L \subseteq \Sigma^*$. Note que el lenguaje puede ser finito o infinito.

Máquina de Turing

Sobre un alfabeto Σ podemos definir lenguajes. Sin embargo, necesitamos una entidad que los procese. En 1936, Alan Turing concibió un modelo matemático abstracto conocido como la **Máquina de Turing**. Este dispositivo consta de una *cinta de memoria infinita* dividida en casillas (donde se escriben símbolos del alfabeto) y un *cabezal* que lee, escribe y se desplaza casilla a casilla.

¿Por qué es relevante? Cualquier computadora moderna —sin importar su arquitectura— es equivalente en potencia de cálculo a una Máquina de Turing. La máquina procesa cadenas de un lenguaje de entrada (por ejemplo, el lenguaje binario $L \subseteq \{0, 1\}$) ejecutando pasos mecánicos finitos para transformar o reconocer dicha información.

Para una presentación formal de la máquina de Turing, consulte el texto *Teoría de la Computación* de Rodrigo de Castro Korgi. Igualmente, para una discusión divulgativa, puede asistir este [video](#).

Antes de terminar, quiero mencionarles que, desgraciadamente, aunque Alan Turing fue un genio de las matemáticas y sus ideas tuvieron un gran impacto, fue perseguido y condenado por su orientación sexual, siendo sometido a una castración química. Finalmente, se suicidó a los 41 años. En la película [The Imitation Game](#) (2014) se narra su historia y su contribución decisiva en la Segunda Guerra Mundial.

Sistema binario

El computador, en su nivel más fundamental, es un sistema electrónico compuesto por millones de pequeños transistores. Estos componentes solo entienden dos estados físicos opuestos: hay corriente o no la hay, está encendido o está apagado (*on/off*), sí o no, blanco o negro. Para procesar esta información numéricamente, representamos abstraídamente estos dos estados mediante los dígitos cero (0) y uno (1). Decimos entonces que el sistema funciona bajo el **sistema binario**, cuyo alfabeto fundamental está definido por:

$$\Sigma = \{0, 1\}$$

Cada uno de estos dígitos dentro del sistema se denomina un **bit** (*binary digit*) y representa la unidad mínima de información en computación clásica.

Con el alfabeto binario podemos formar cadenas de cualquier longitud. Sin embargo, centraremos nuestra discusión en la cadena de 8 bits, por ejemplo 01011110. A esta secuencia se le llama **byte**. El término correcto es octeto, pero se termina imponiendo el término en inglés.

Supongamos que tenemos la cadena 11111111 (todos prendidos), podemos tener

$$\begin{array}{cccccccc} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow \\ 2^7 & 2^6 & 2^5 & 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \end{array}$$

Se tienen entonces $128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255$ significados diferente. Si todos están apagados $\rightarrow 00000000 = 0$. Luego, tenemos el rango $[0, 255]$ de valores diferentes entre apagado y prendido. Cualquiera de esos valores puede ser representado por un byte. Note que $2^8 = 256$.

¿Qué significa entonces la cadena 01100110 para la computadora? No tenemos una respuesta única para esta pregunta. Depende de las convenciones. Puede ser una letra, un número o lo que sea.

Pensemos primero en números. Tal vez en nuestro día a día no lo tengamos en cuenta, pero usamos una convención para representar los números: el sistema decimal. El alfabeto decimal está dado por $\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$. De tal forma que la cadena 40_{10} representa el número 40, pues

$$40_{10} = 4 \times 10^1 + 0 \times 10^0$$

Así, el sistema decimal es el que usamos todos los días, tal vez sin pensar en esas reglas. Por ejemplo el número 86409

$$86409_{10} = 8 \times 10^4 + 6 \times 10^3 + 4 \times 10^2 + 0 \times 10^1 + 9 \times 10^0$$

Ahora bien, si queremos representar ese mismo número 40 pero en su sistema binario con cadenas de longitud 8, ¿qué hacemos? Dividimos por 2

Ejemplo: Vamos a representar por ejemplo el número 40 en binario de 8 bits:

$$\begin{array}{ll} \frac{40}{2} = 20 & \text{con residuo } 0 \\ \frac{20}{2} = 10 & \text{con residuo } 0 \\ \frac{10}{2} = 5 & \text{con residuo } 0 \\ \frac{5}{2} = 2 & \text{con residuo } 1 \\ \frac{2}{2} = 1 & \text{con residuo } 0 \\ \frac{1}{2} = 0 & \text{con residuo } 1 \end{array}$$

Luego, tomamos los residuos en orden inverso $40 = 101000$. Pero esta cadena es seis bits y la necesitamos de 8 bits. Para ello, agregamos ceros a la izquierda, es decir, $40 = 00101000$. Note que

$$\begin{aligned} 00101000 &= 0 \times 2^7 + 0 \times 2^6 + 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 \\ &= 0 + 0 + 32 + 0 + 8 + 0 + 0 + 0 = 40 \end{aligned}$$

Ejemplo:

$$4 = 00000100; \quad 3 = 00000011; \quad 11 = 00001011$$

En la [Tabla 1](#) se muestran algunos números enteros positivos en la representación decimal y binario de 8 bits, separadas de a 4 bits (Nibble).

Cantidad	Representación decimal	Representación binaria (8 bits)
	0	0000 0000
	1	0000 0001
	2	0000 0010
	3	0000 0011
	4	0000 0100
	5	0000 0101
	6	0000 0110
	7	0000 0111
	8	0000 1000
	9	0000 1001
	10	0000 1010
	11	0000 1011
	12	0000 1100
	13	0000 1101
	14	0000 1110
	15	0000 1111

Tabla 1: En el sistema numérico binario de 8 bits, cada número entero está representado por un patrón de un byte (8 dígitos binarios).

Supongamos ahora que queremos escribir los números enteros negativos. Con 8 bits, podemos representar desde el 0 hasta el 255, pero todos positivos. Si queremos los negativos, entonces nos quedarían solo 128, es decir, $[-127, 127]$ (un bit para el cero). Por ejemplo, para representar el -5 , tenemos la opción más simple, usar el primer bit para el signo, 0 para positivo y 1 para negativo. Esto es $5 = 00000101$ y $-5 = 10000101$. Sin embargo, esto tiene un problema: existen dos cero $0 = 00000000$ y $-0 = 10000000$. La solución a este problema es la siguiente: tome el $5 = 00000101$ y lo niegas 11111010 y luego le sumas el 1 en binario

$$\begin{array}{r}
 11111010 \\
 +00000001 \\
 \hline
 11111011
 \end{array}$$

El resultado obtenido será el número -5 .

Unidades de medida

Recordemos que 1 byte son 8 bits. Por otro lado, el prefijo **kilo** significa 1000. Pero las computadoras trabajan con potencias de 2.

$$512 \text{ bytes} = 2^9 \text{ bytes}$$

$$1 \text{ Kilobyte} = 1 \text{ KB} = 2^{10} \text{ bytes} = 1024 \text{ bytes}$$

$$1 \text{ Megabyte} = 1 \text{ MB} = 2^{20} \text{ bytes} = 1024 \text{ KB} = 1\,048\,576 \text{ bytes}$$

$$1 \text{ Gigabyte} = 1 \text{ GB} = 2^{30} \text{ bytes} = 1024 \text{ MB}$$

$$1 \text{ Terabyte} = 1 \text{ TB} = 2^{40} \text{ bytes} = 1024 \text{ GB}$$

A finales de los años 70 comenzaron a popularizarse las computadoras de 16 bits. Un ejemplo muy conocido de esta generación (aunque apareció algunos años después, en 1990) fue el Super Nintendo, cuya CPU era de 16 bits. ¿Cuántos números enteros diferentes puede representar un computador de 16 bits?

Cada bit toma dos valores distintos: 0 o 1 $\Rightarrow 2^{16} = 65536$

Lo que significa que hay 65536 patrones distintos

$\Rightarrow$ La mitad toma valores negativos y la otra mitad valores positivos 32768

$\Rightarrow$ hay 32768 números negativos, desde -32768 hasta -1

$\Rightarrow$ hay 32768 números positivos, desde 0 hasta 32767 $\Rightarrow [-32768, 32767]$

Como el número de direcciones diferentes es: 65536, se tiene 65536 bytes = 64 Kilobytes

Los símbolos

Ya sabiendo como se representan los números enteros. Nos preguntamos entonces como el computador entiende las letras y los símbolos. Por ejemplo, la palabra Hola. Para ello, inicialmente se creó el código ASCII (*American Standard Code for Information Interchange*)

Supongamos que quiero escribir “Hola”. A la letra H le corresponde el número decimal 72, a la o le corresponde el 111, a la l le corresponde el 108 y a la a le corresponde el 97. Convertimos cada número decimal a binario y obtenemos la siguiente representación en binario de 8 bits:

$$H \leftrightarrow 72 \Rightarrow 01001000 : 0 \times 2^7 + 1 \times 2^6 + 0 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 0 \times 2^0$$

$$o \leftrightarrow 111 \Rightarrow 01101111 : 0 \times 2^7 + 1 \times 2^6 + 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

$$l \leftrightarrow 108 \Rightarrow 01101100 : 0 \times 2^7 + 1 \times 2^6 + 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0$$

$$a \leftrightarrow 97 \Rightarrow 01100001 : 0 \times 2^7 + 1 \times 2^6 + 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

Así, en binario de 8 bits, la palabra “Hola” se escribe como (note que la H mayúscula y la h minúscula tienen diferente asignación): 01000000 01101111 01101100 01100001.

Si prestamos atención a la tabla, podemos notar que es excluyente, diseñada, como su nombre lo dice, para incluir solamente caracteres de la lengua inglesa. Por ejemplo, piensa en la letra ñ. Escribir niño en el computador, hace algunos años era una tortura, así como π o las tildes. Además, ¿Qué ocurre entonces si deseamos escribir en ruso? ¿O mandarín? o utilizar otros símbolos que no estén allí. Para resolver esto se crea Unicode y posteriormente UTF-8.

Tabla 2: Código ASCII en base decimal. Note que fue diseñada para 7 bits, es decir, números entre 0 y 127.

0 nul	1 soh	2 stx	3 etx	4 eot	5 enq	6 ack	7 bel
8 bs	9 ht	10 nl	11 vt	12 np	13 cr	14 so	15 si
16 dle	17 dc1	18 dc2	19 dc3	20 dc4	21 nak	22 syn	23 etb
24 can	25 em	26 sub	27 esc	28 fs	29 gs	30 rs	31 us
32 sp	33 !	34 ”	35 #	36 \$	37 %	38 &	39 ’
40 (	41)	42 *	43 +	44 ,	45 -	46 .	47 /
48 0	49 1	50 2	51 3	52 4	53 5	54 6	55 7
56 8	57 9	58 :	59 ;	60 <	61 =	62 >	63 ?
64 @	65 A	66 B	67 C	68 D	69 E	70 F	71 G
72 H	73 I	74 J	75 K	76 L	77 M	78 N	79 O
80 P	81 Q	82 R	83 S	84 T	85 U	86 V	87 W
88 X	89 Y	90 Z	91 [	92 \	93]	94 ^	95 _
96 ‘	97 a	98 b	99 c	100 d	101 e	102 f	103 g
104 h	105 i	106 j	107 k	108 l	109 m	110 n	111 o
112 p	113 q	114 r	115 s	116 t	117 u	118 v	119 w
120 x	121 y	122 z	123 {	124	125 }	126 ~	127 del

Representación de un número flotante (float)

Hasta el momento, hemos centrado la discusión en los números enteros. ¿Qué pasa entonces si queremos representar magnitudes reales o muy grandes, como la velocidad de la luz, $c = 2.99792458 \times 10^8 \text{ m s}^{-1}$?

Primero, notemos que una misma cantidad puede ser expresada de diferentes formas equivalentes; por ejemplo, $c = 0.299792458 \times 10^9 \text{ m s}^{-1}$, donde hemos desplazado el punto hacia la izquierda y ajustado la potencia de 10. Por esta flexibilidad para desplazar la posición decimal, este esquema se denomina **representación de punto flotante**.

En esta representación:

- El conjunto de dígitos significativos se conoce como **mantisa** (o fracción).
- La potencia a la que se eleva la base determina el **exponente**.
- El **signo** indica si el número es positivo o negativo.

Esta concatenación de información (Signo, Exponente y Mantisa) es lo que la computadora almacena en memoria. Dado que el hardware no puede guardar infinitos dígitos reales, su representación en el computador es siempre finita y discreta; a estos números representables los llamamos **números de máquina**. Si un cálculo excede el rango representable por la arquitectura, se producen dos tipos de límites de precisión:

- **Overflow (desbordamiento):** Ocurre cuando la magnitud de un número es mayor que el valor máximo que el hardware puede almacenar. En este caso, el sistema retorna infinito ($\pm\infty$).
- **Underflow (subdesbordamiento):** Ocurre cuando la magnitud de un número es tan cercana a cero que no es posible representarlo con la precisión del sistema, por lo que el hardware lo aproxima automáticamente a cero (0.0).

- a) *Parte entera* (13_{10}): Dividimos sucesivamente entre 2 guardando el cociente y anotando el residuo hasta llegar a cociente 0:

$$\begin{aligned}13/2 &= 6 && \text{con residuo } 1 \\6/2 &= 3 && \text{con residuo } 0 \\3/2 &= 1 && \text{con residuo } 1 \\1/2 &= 0 && \text{con residuo } 1\end{aligned}$$

Luego, tomamos los residuos en orden inverso (del último al primero) para obtener la representación binaria $13 = 00001101$

- b) *Parte fraccionaria* (0.625_{10}): Multiplicamos la parte decimal por 2 repetidamente. La parte entera del resultado será el dígito binario (0 o 1), y continuaremos multiplicando solo la parte fraccionaria resultante hasta que la parte fraccionaria llegue a cero o hasta alcanzar la precisión deseada:

$$\begin{aligned}0.625 \times 2 &= \mathbf{1.25} \implies \text{Extraemos el } \mathbf{1} && (\text{posición } 2^{-1} = 0.5) \\0.250 \times 2 &= \mathbf{0.50} \implies \text{Extraemos el } \mathbf{0} && (\text{posición } 2^{-2} = 0.25) \\0.500 \times 2 &= \mathbf{1.00} \implies \text{Extraemos el } \mathbf{1} && (\text{posición } 2^{-3} = 0.125)\end{aligned}$$

Como la parte fraccionaria llegó a cero (0.00), el proceso termina. Leyendo las partes enteras de arriba hacia abajo: $0.625_{10} = 0.101$. Posteriormente, unimos la parte entera y la fraccionaria para obtener la representación binaria exacta: $|X| = 13.625 = 1101.101$

Paso 3: Normalización en notación científica binaria

Movemos el punto decimal 3 posiciones hacia la izquierda para dejar un solo 1 a la izquierda del punto ($1.f \times 2^e$): $1101.101 = 1.101101 \times 2^3$. Así, identificamos:

- **Exponente real** (e): $e = 3$.
- **Mantisa o Fracción** (f): La secuencia que queda a la derecha del punto es .101101.

Paso 4: Cálculo del Exponente con Sesgo (C) en 11 bits

Para representar exponentes positivos y negativos, le sumamos el sesgo (*bias*) de 1023 correspondiente a 64 bits:

$$C = e + 1023 = 3 + 1023 = \mathbf{1026_{10}}$$

Ahora convertimos el número decimal 1026 a un bloque binario de 11 bits mediante divisiones por 2:

$$\begin{array}{ll} 1026/2 = 513 & \text{residuo } 0 \\ 513/2 = 256 & \text{residuo } 1 \\ 256/2 = 128 & \text{residuo } 0 \\ 128/2 = 64 & \text{residuo } 0 \\ 64/2 = 32 & \text{residuo } 0 \\ 32/2 = 16 & \text{residuo } 0 \\ 16/2 = 8 & \text{residuo } 0 \\ 8/2 = 4 & \text{residuo } 0 \\ 4/2 = 2 & \text{residuo } 0 \\ 2/2 = 1 & \text{residuo } 0 \\ 1/2 = 0 & \text{residuo } 1 \end{array}$$

Leyendo de abajo hacia arriba obtenemos la secuencia de 11 bits: $C = 1026 = 10000000010$

Paso 5: Ensamblar la cadena final de 64 bits

- **Bit de signo (S , 1 bit):** 1
- **Exponente (C , 11 bits):** 10000000010
- **Mantisa (f , 52 bits):** Tomamos los 6 bits significativos obtenidos en la normalización (101101) y rellenamos los 46 bits restantes con ceros a la derecha:

[illegible]

Resultado final en memoria de la computadora:

[illegible]

Pensemos ahora por ejemplo en un procesador de 64 bits, cual es el máximo y el mínimo valor que puede representar.

Límites de la representación (Doble precisión - 64 bits)

1. El Número Máximo Posible (Límite de overflow) Para obtener el valor finito más grande posible, configuramos:

- **Signo** ($S = 0$): Positivo.
- **Exponente** ($C = 2046$): El mayor valor permitido antes de ∞ . Note que la cadena 1111111111 (2047) está reservada por el hardware para avisar que ocurrió una división por cero (+Inf, -Inf) o un cálculo indeterminado (NaN).

Problemas:

1. Consulta qué otros sistemas de numeración se usan y cuáles ya no se usan. En particular, consulta el sistema de numeración usado por la cultura Maya.
2. Escriba el número -15 en formato binario de 8 bits.
3. Escriba el número 57 en formato binario de 8 bits.
4. ¿Cuál son los límites de overflow y de underflow para una computadora con 16/bits y con 32/bits?
5. ¿Cuáles son los límites de overflow y de underflow para una computadora con 16/bits y con 32/bits?
6. ¿Cuántos bits se necesitan para representar los números del 0 al 18, ambos inclusive?
7. Escriba su nombre completo en binario.
8. Obtenga la cadena binaria completa de 64 bits (Signo S , Exponente C y Mantisa f) bajo el estándar IEEE 754 para el número real: $X = -0.1$.
9. Los sistemas de cómputo utilizan cadenas binarias para identificar (direccionar) cada posición de memoria RAM. Suponga que a cada dirección le corresponde exactamente 1 byte de información.
 - a) En un sistema de 16 bits (como la consola Super Nintendo o las primeras PC), ¿cuántas direcciones distintas se pueden generar y cuál es la memoria RAM máxima (en Kilobytes) que se puede direccionar?
 - b) En un sistema de 32 bits, calcule cuántas direcciones distintas se generan y demuestre por qué la memoria RAM máxima que puede direccionar este sistema es de exactamente 4 GB.
10. En un experimento de física computacional se evalúa la función $f(x) = \sqrt{x+1} - \sqrt{x}$ para un valor muy grande $x = 10^{16}$.
 - a) Muestre por qué la evaluación directa de $f(x)$ en precisión doble (IEEE 754) produce pérdida severa de dígitos significativos (cancelación catastrófica).
 - b) Proponga una reformulación algebraica de $f(x)$ que evite este problema numérico y calcule el valor numérico estable que la computadora obtendría.